Resala Charity

UML class diagrams & design patterns

Architecture of the current codebase, module by module — every diagram generated from source.

SOURCE OF TRUTH — src/main/java · 132 files
Compiled UML rendered with d2 · November 2026

HOW TO READ THESE DIAGRAMS

Notation & legend

Edges

——————▷
extends / inherits (solid line, hollow triangle)

- - - - ▷
implements (dashed line, hollow triangle)

——————◇
wraps / composes (decorator, aggregate)

——————→
uses / association

Member markers

\+ public – private # protected

«abstract» abstract class
«interface» interface
«record» immutable value carrier
«enum» enumerated type
«singleton» one instance per JVM

Containers

· Package = labelled container (e.g. com.resala.donation)

· Dashed teal border = «pattern» boundary — participants of one design pattern

· Navy header, white text = class; white header rows below list members then methods

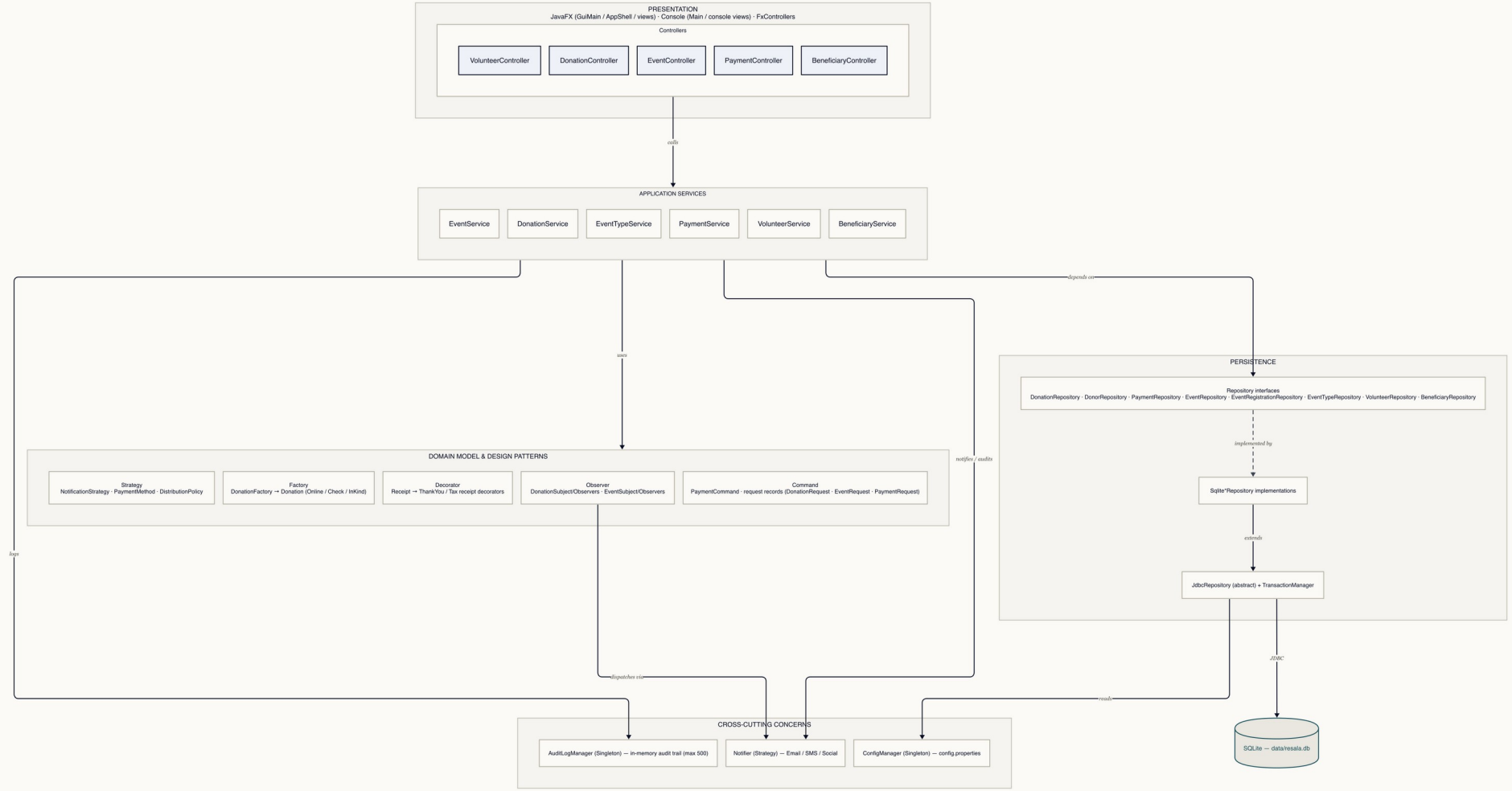
· Each count: “Module k/9” and “Pattern k/13” on its own slide

A note on fidelity

Member names and types are transcribed from the source. Where the code ships dead weight — e.g. the unused com.resala.volunteer.Volunteer duplicate of model.Volunteer — it is drawn but marked «unused», not hidden.

SYSTEM AT A GLANCE

The layers, one picture



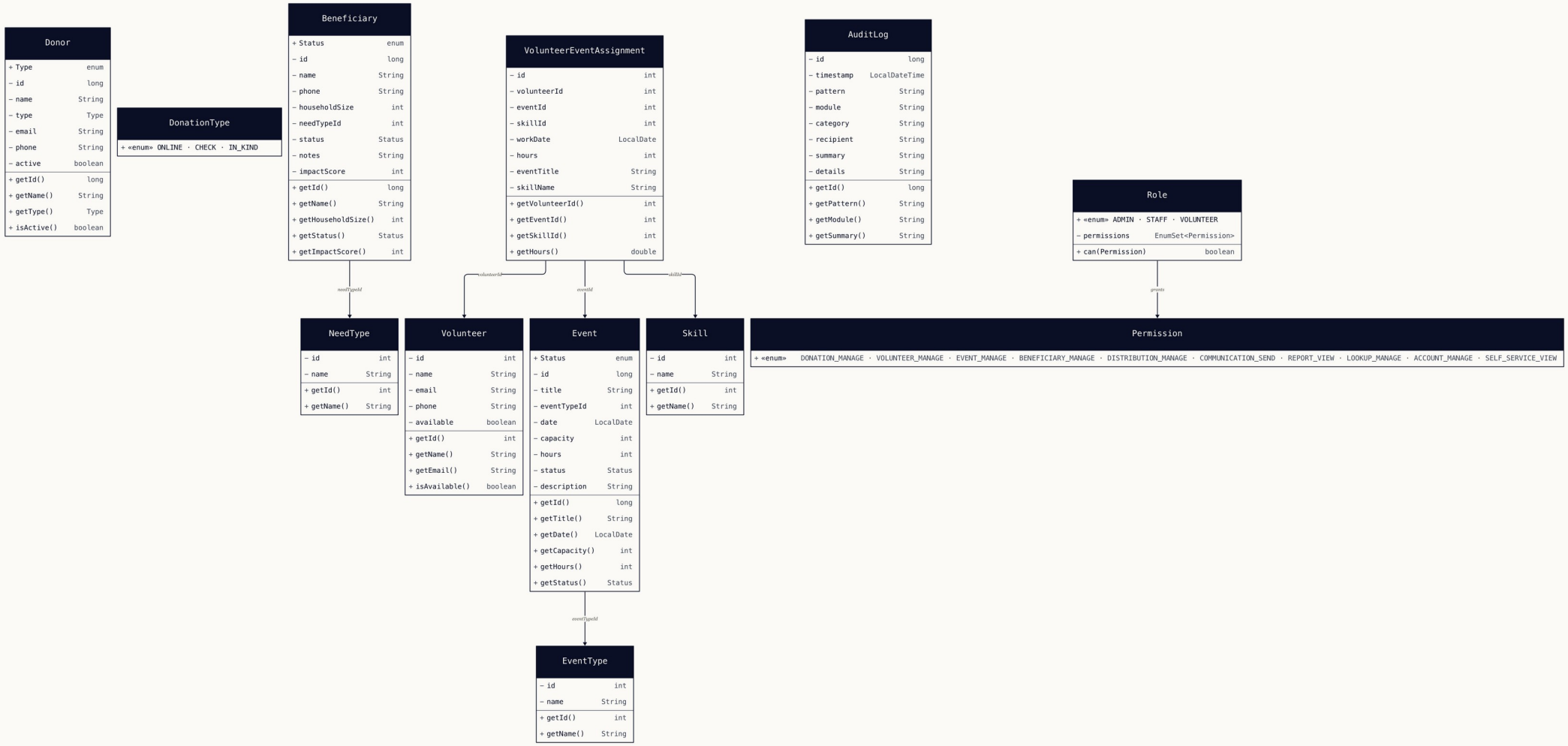
Presentation → controllers → services → domain patterns → persistence → SQLite

DESIGN PATTERNS AT A GLANCE

All thirteen, as they appear in code

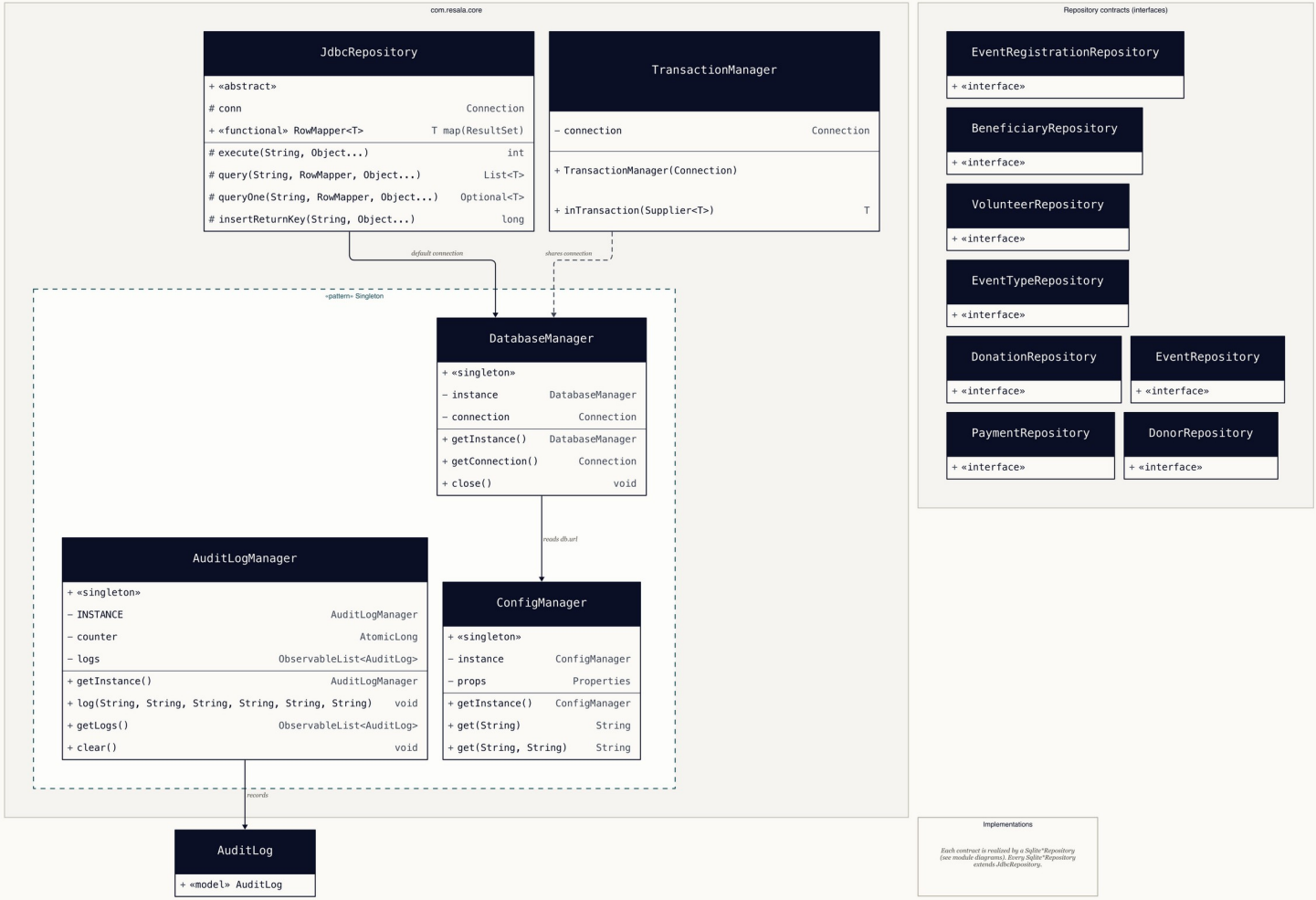
Pattern	Where	Key types
Singleton	core	ConfigManager · DatabaseManager · AuditLogManager
Factory Method	donation	DonationFactory ⟶ Online · Check · InKind
Abstract Factory	view	UiFactory ⟶ ConsoleUiFactory · JavaFxUiFactory
Strategy — 3 families	payment · notification · beneficiary	PaymentMethod · NotificationStrategy · DistributionPolicy
Observer — 2 subjects	notification	DonationSubject · EventSubject · FeedObserver (dual)
Decorator	donation	Receipt ⟵ Tax / ThankYou decorators
Adapter	payment	FakePaymentGatewayAdapter ⟶ FakePaymentProvider
Command	payment	PaymentCommand ⟶ PaymentService ⟶ gateway
State	payment	PaymentStatus (PENDING ⟶ PAID / FAILED)
Repository	persistence	8 contracts + JdbcRepository base · Sqlite* impls
Unit of Work	core · payment	TransactionManager.inTransaction(…)
Static factory	payment	Payment.createPending / restore · PaymentMethods.fromCode
Value objects	everywhere	Records: DonationRequest, PaymentRequest, GatewayResult…

The domain vocabulary



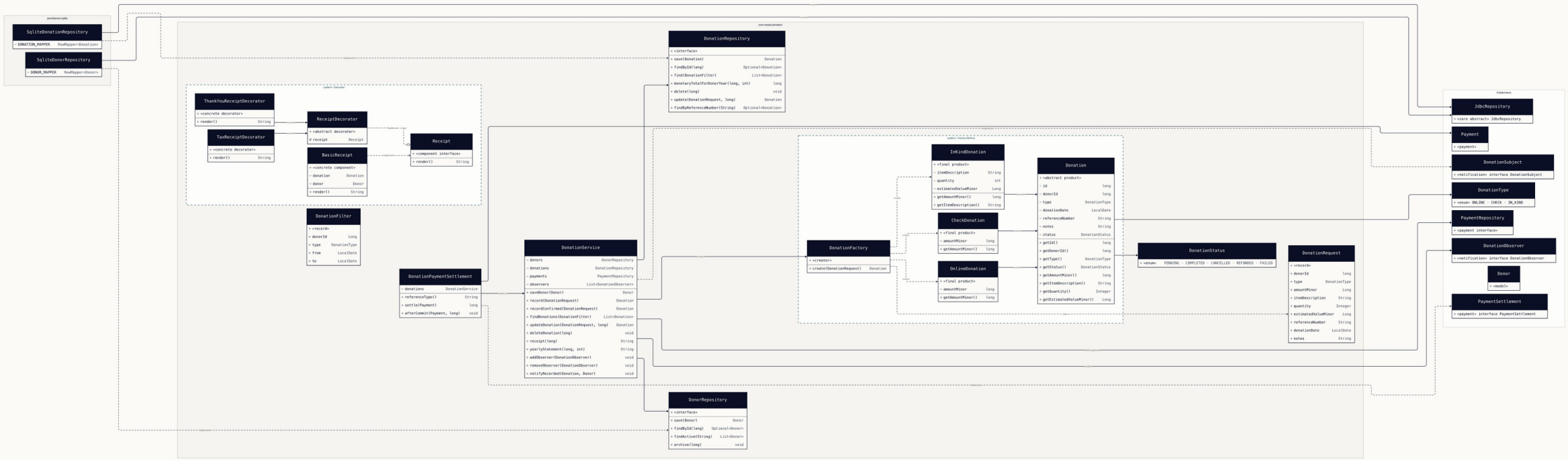
com.resala.model — entities, enums, lookup tables · 12 classes

One SQLite file, one connection



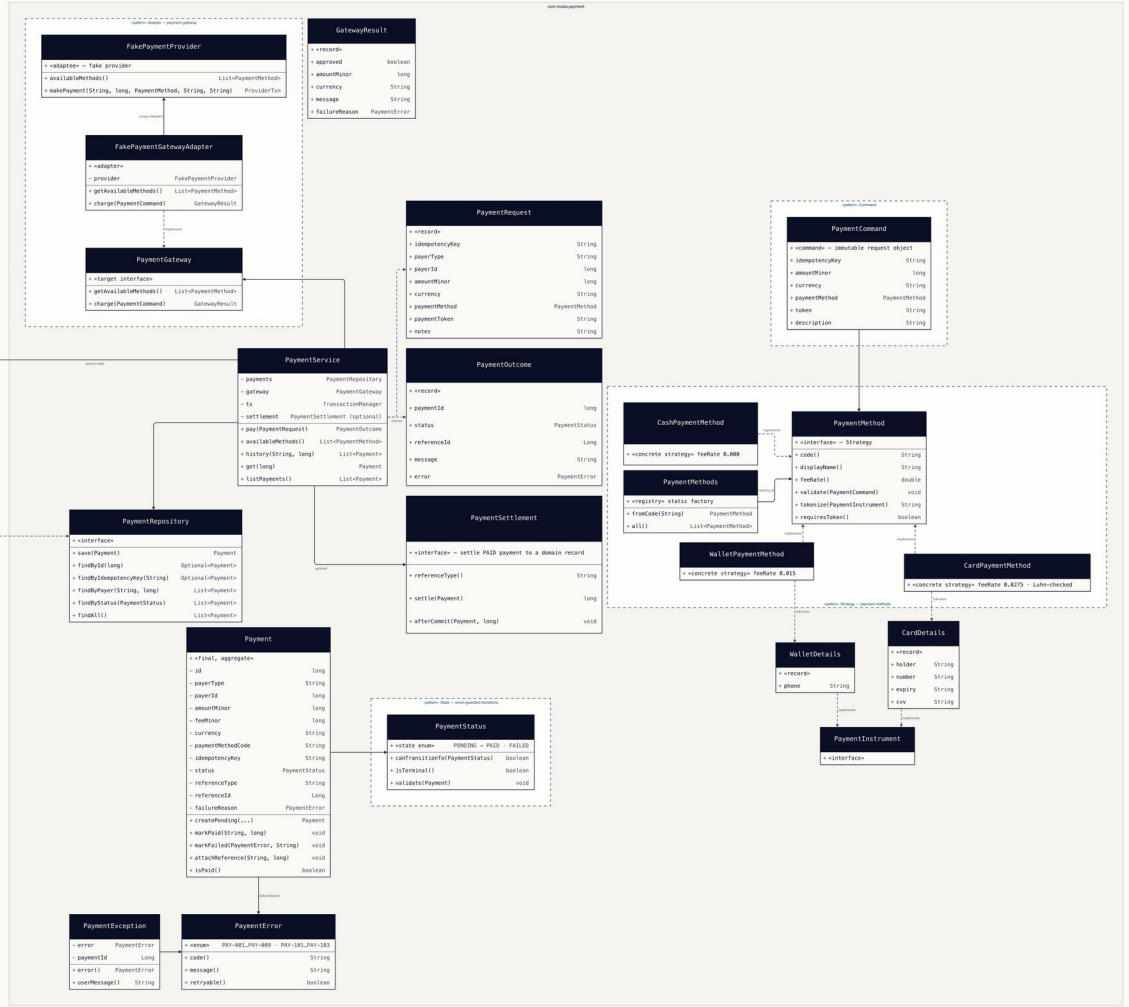
com.resala.core · persistence.sqlite — Singletons · JdbcRepository · contracts

Factory, decorator receipts, observer



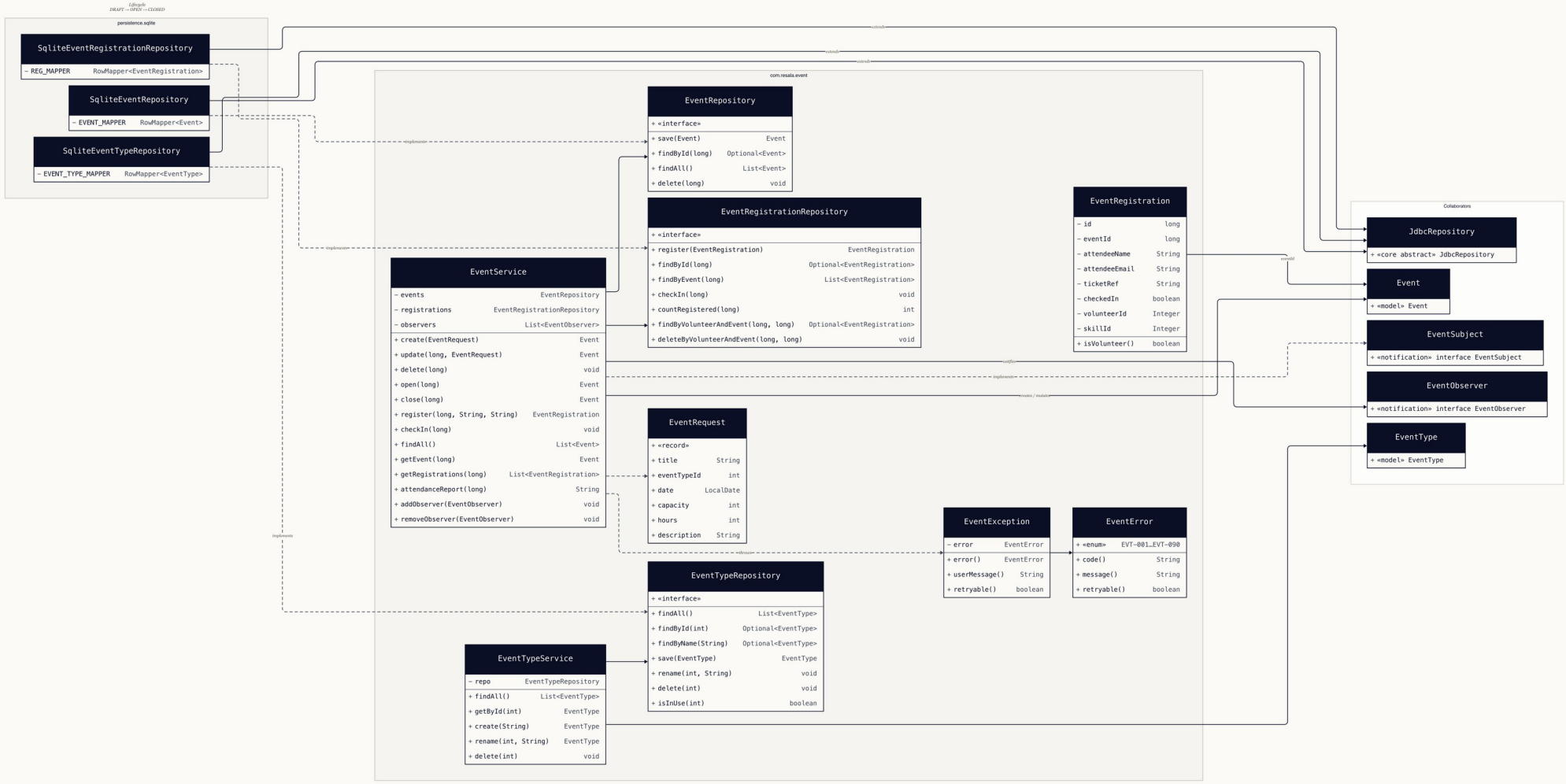
com.resala.donation — Factory Method · Decorator · DonationSubject

Strategy, adapter, idempotent retries



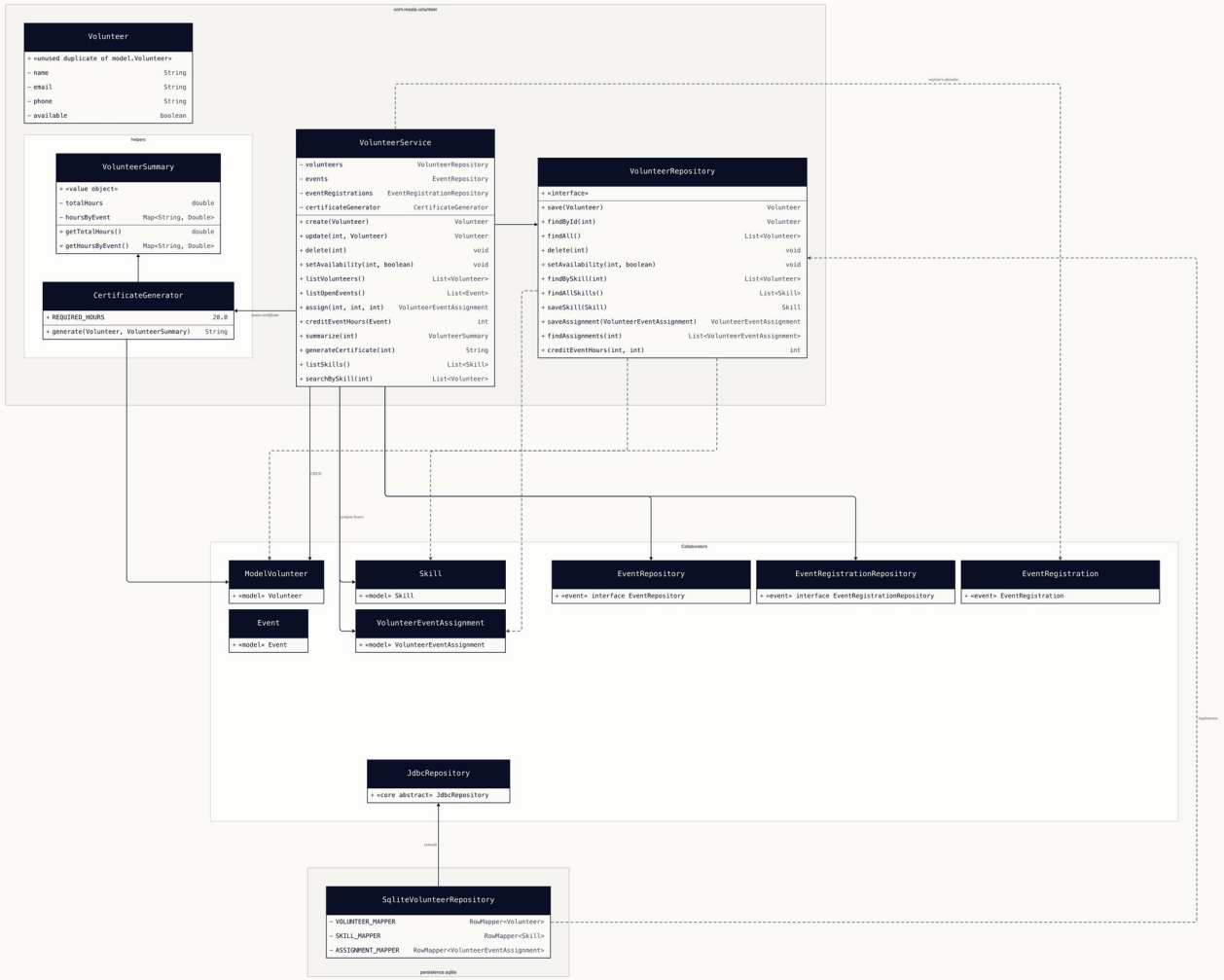
com.resala.payment — Strategy · Adapter · Command · State (enum) · static factory

Lifecycle & attendee notifications



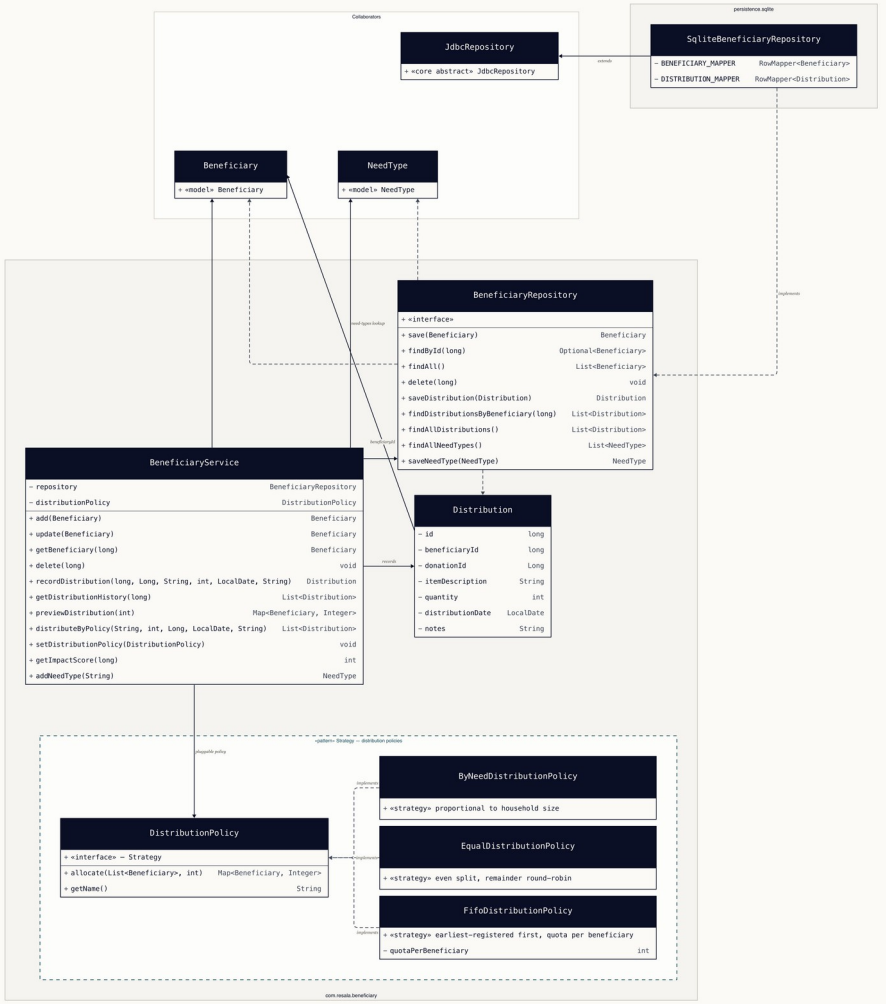
com.resala.event — DRAFT → OPEN → CLOSED · EventSubject

Assignments, skills, certificates



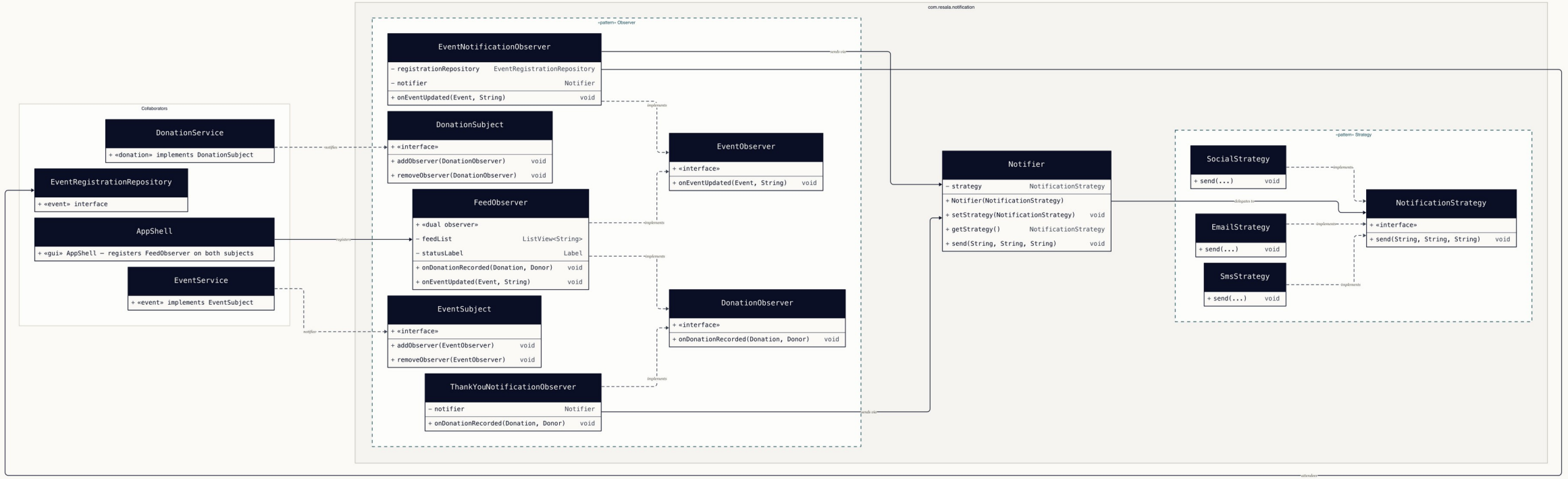
com.resala.volunteer — VolunteerService · CertificateGenerator (20 h threshold)

Swappable distribution policies



com.resala.beneficiary — Strategy: ByNeed · Equal · FIFO

Two subjects, three channels



com.resala.notification — Observer · Strategy · dual FeedObserver

JavaFX shell & console UI



com.resala.controller · com.resala.gui · com.resala.view — controllers · AppShell · views

Design patterns, as the code ships them

Thirteen diagrams. Every pattern found in the source, each inside a dashed «pattern» boundary.

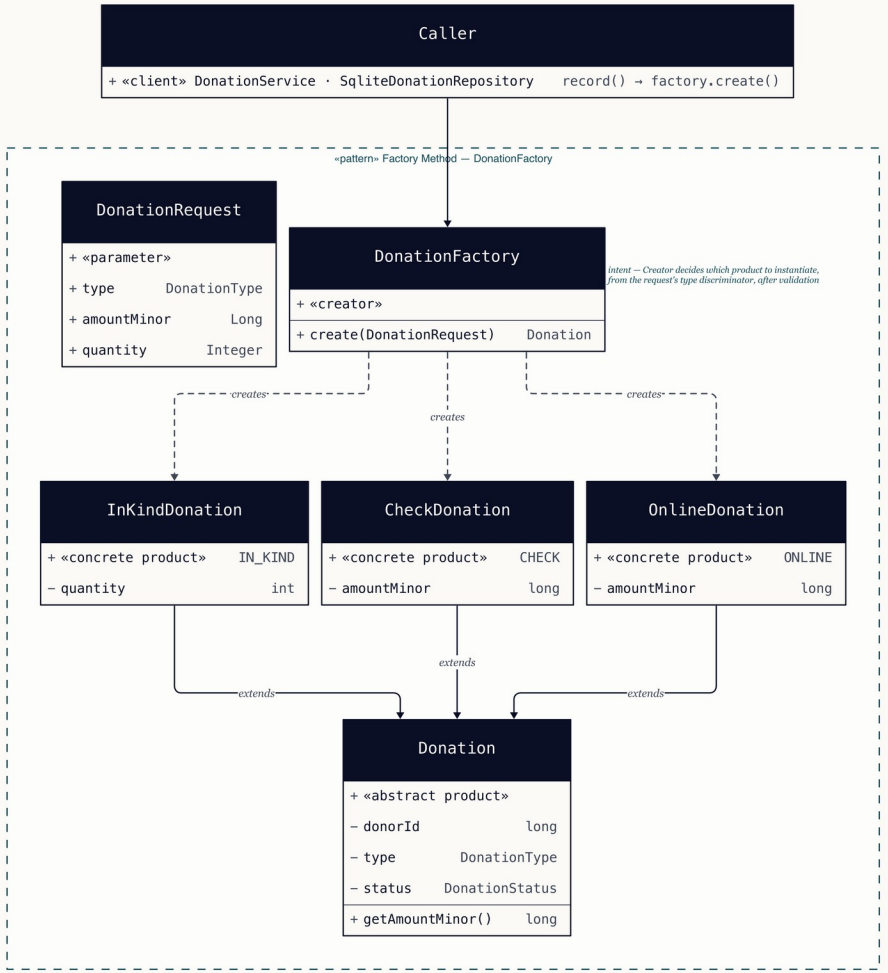
SINGLETON ×3 · FACTORY METHOD · ABSTRACT FACTORY · STRATEGY ×3
OBSERVER ×2 · DECORATOR · ADAPTER · COMMAND · STATE · REPOSITORY · UNIT OF WORK

One instance per JVM



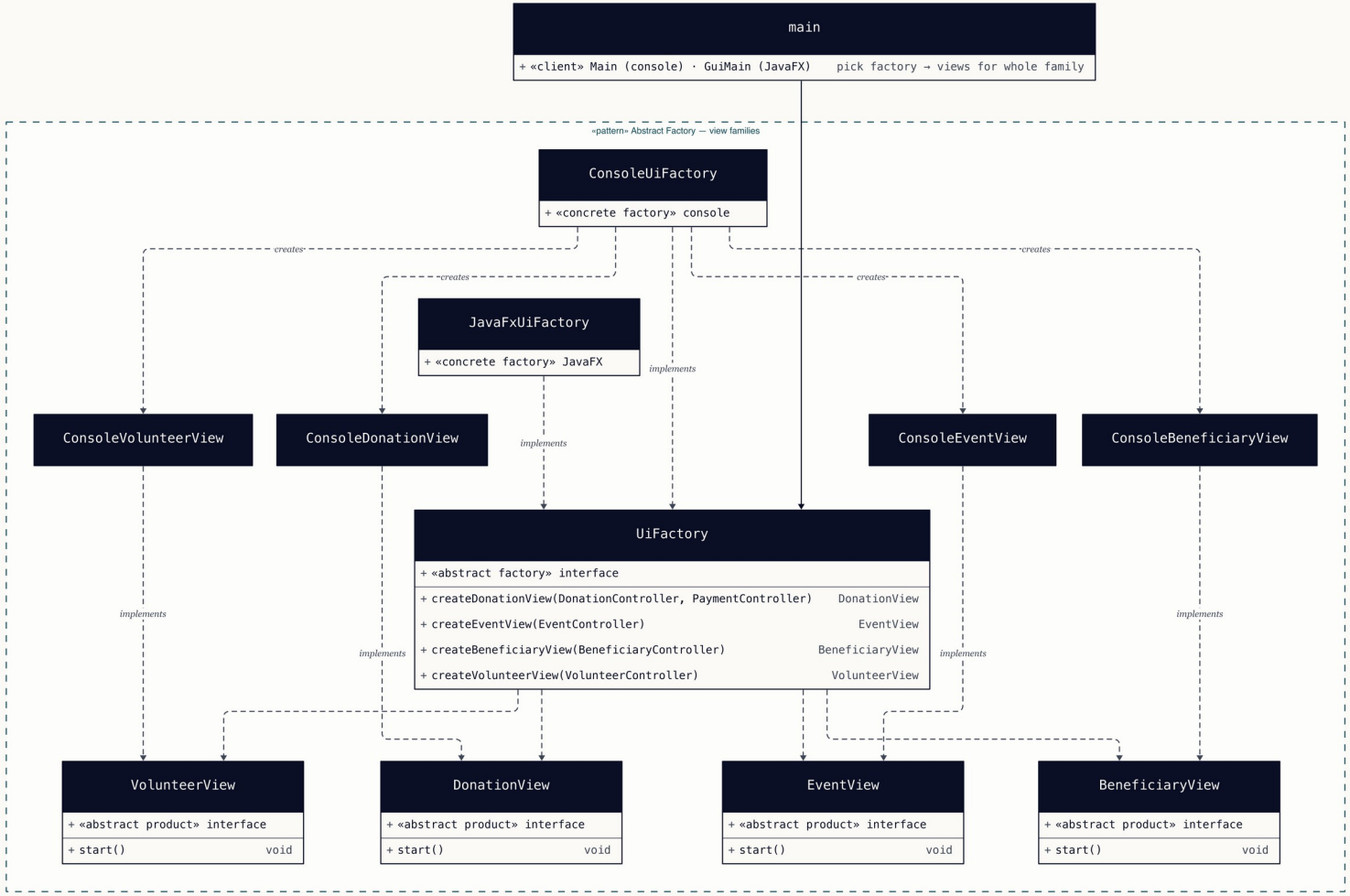
Where — com.resala.core.ConfigManager · DatabaseManager · AuditLogManager

DonationFactory decides the product



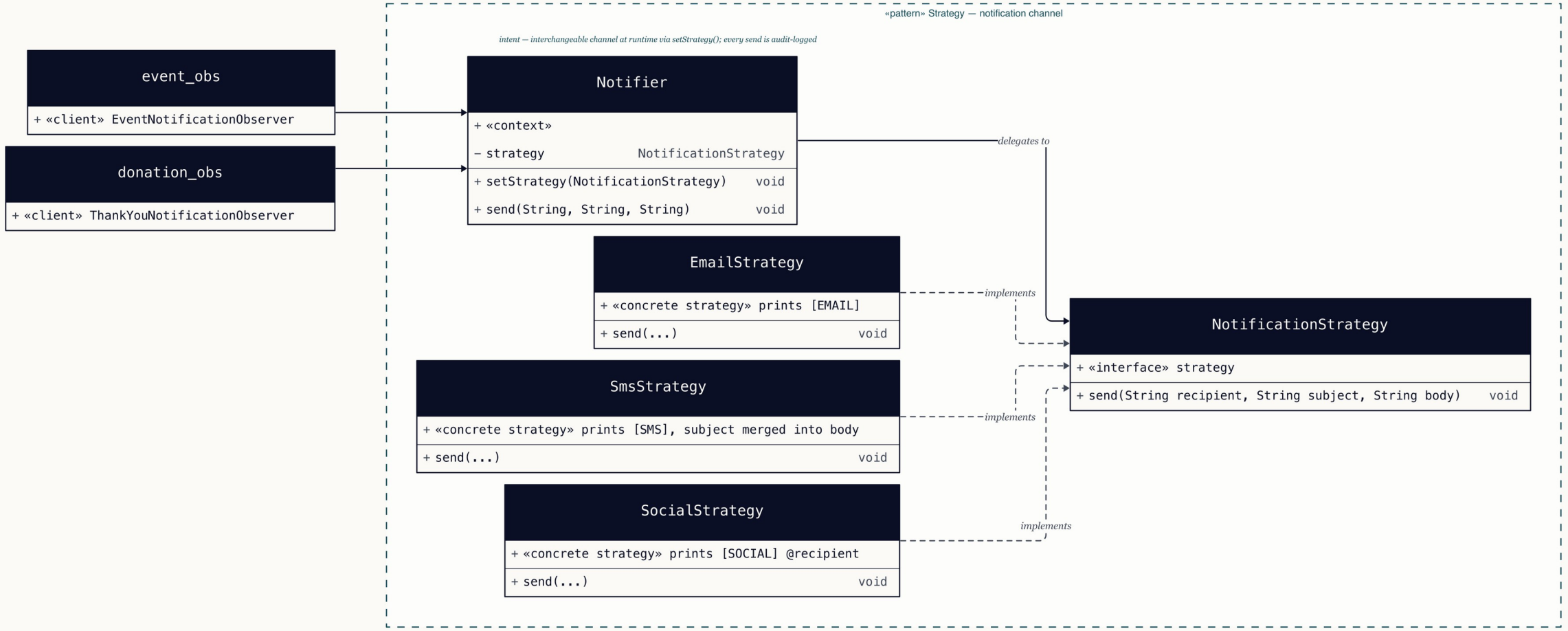
Where — com.resala.donation.DonationFactory (Online · Check · InKind)

A whole family of views, one factory



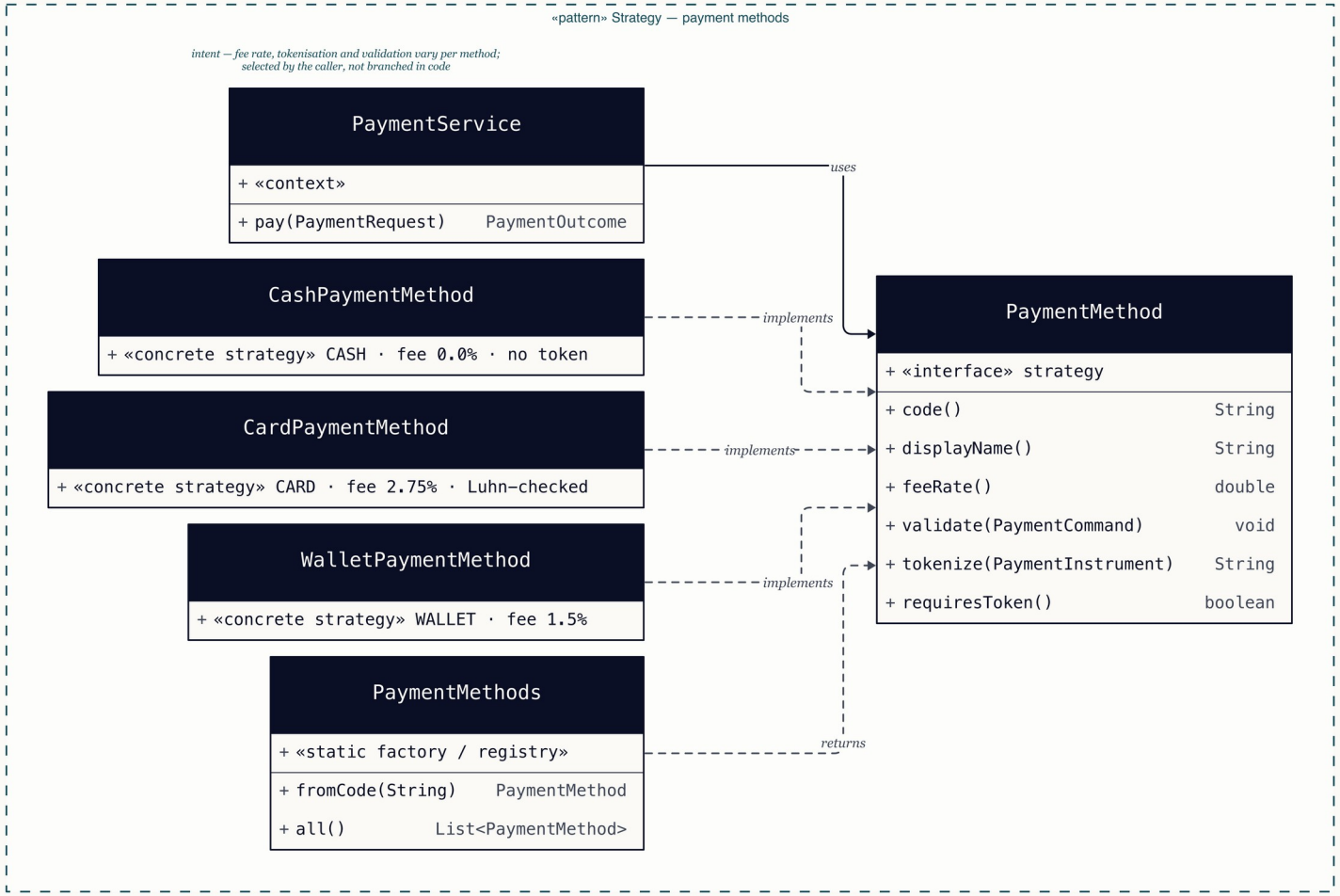
Where — com.resala.view.UiFactory · ConsoleUiFactory · JavaFxUiFactory

Channels swappable at runtime



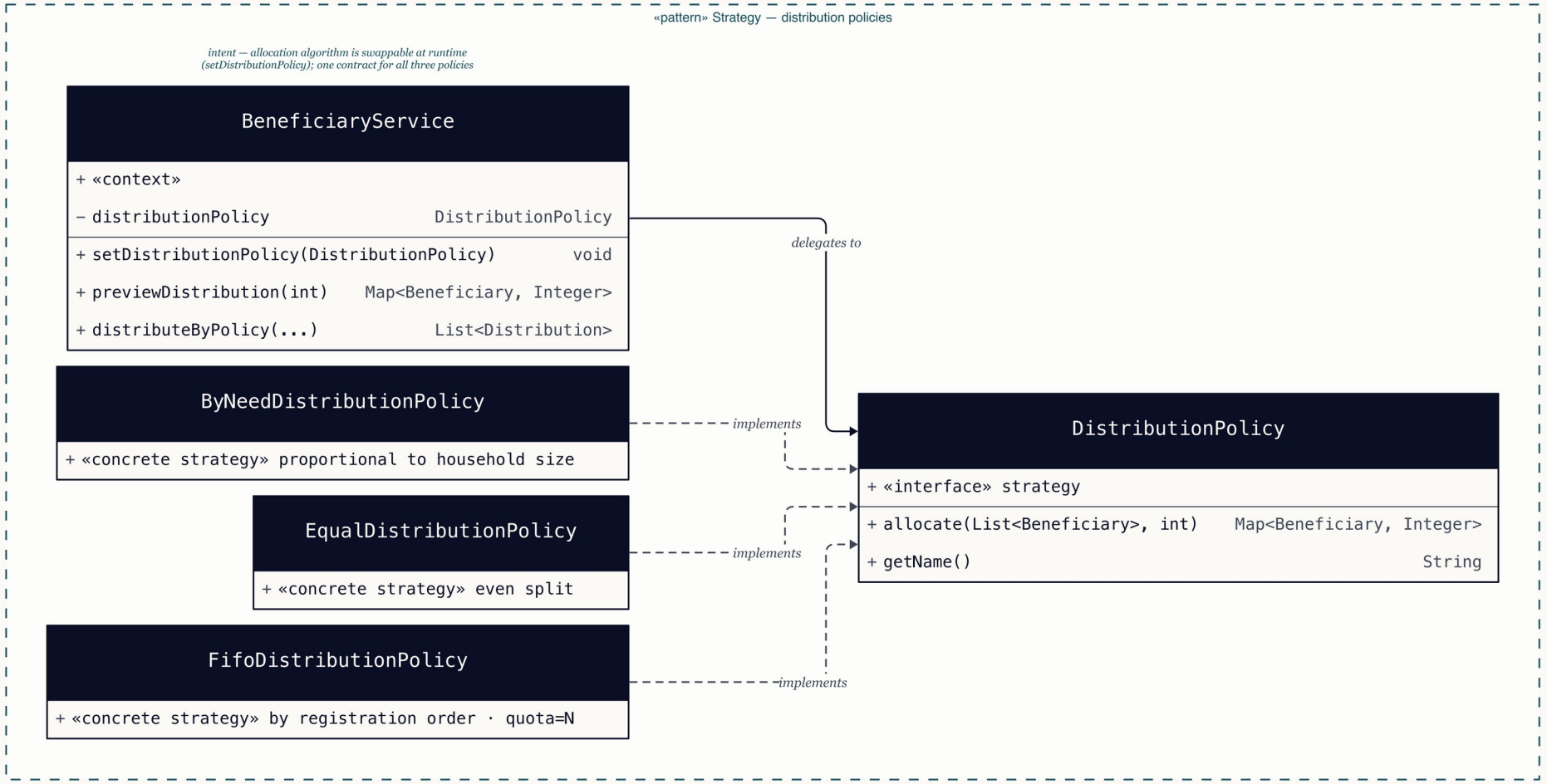
Where — com.resala.notification.Notifier · Email / Sms / SocialStrategy

Cash, card, wallet — same contract



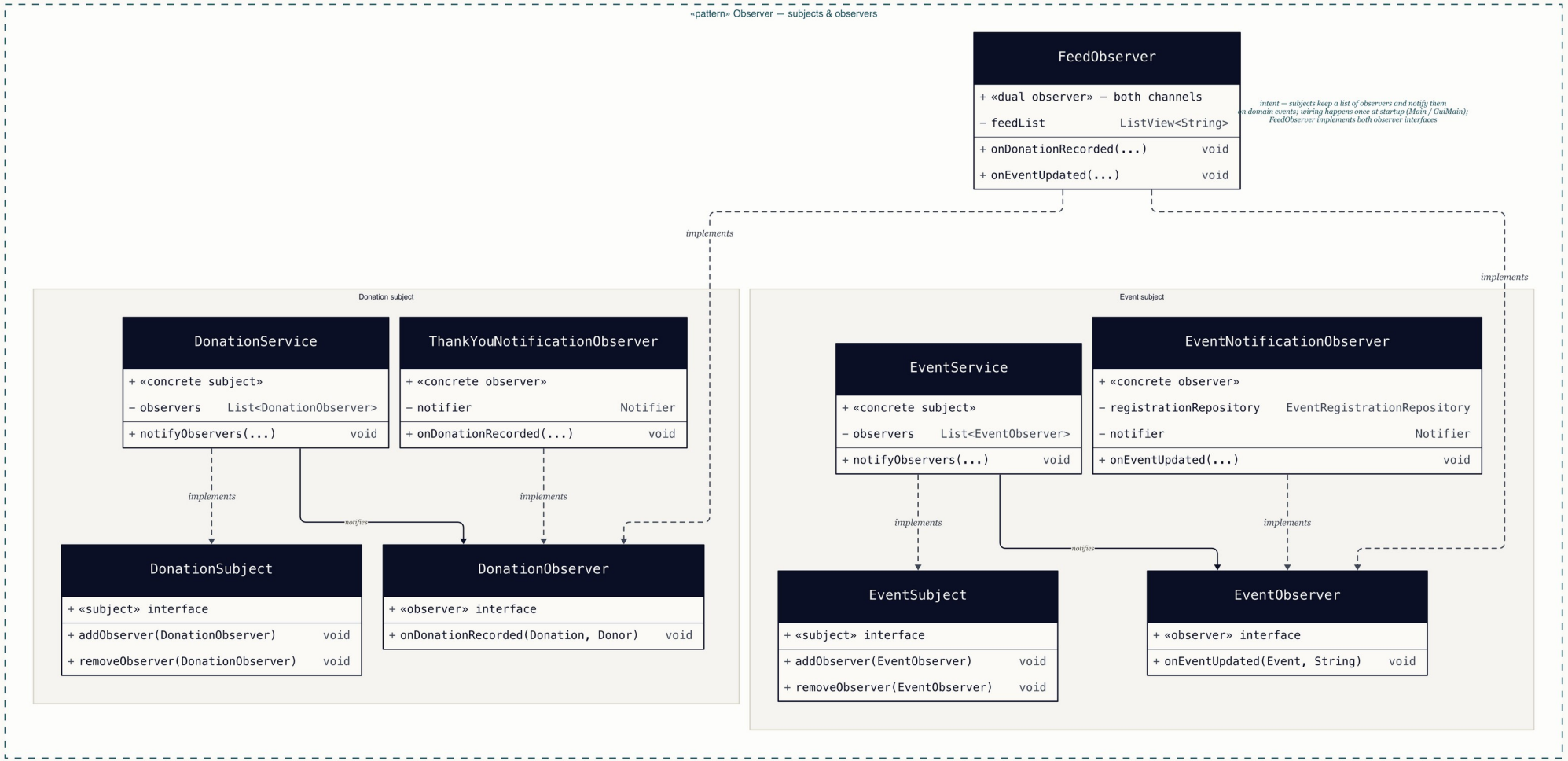
Where — com.resala.payment.PaymentMethod · Cash / Card / WalletPaymentMethod · PaymentMethods

Three allocation policies, one call



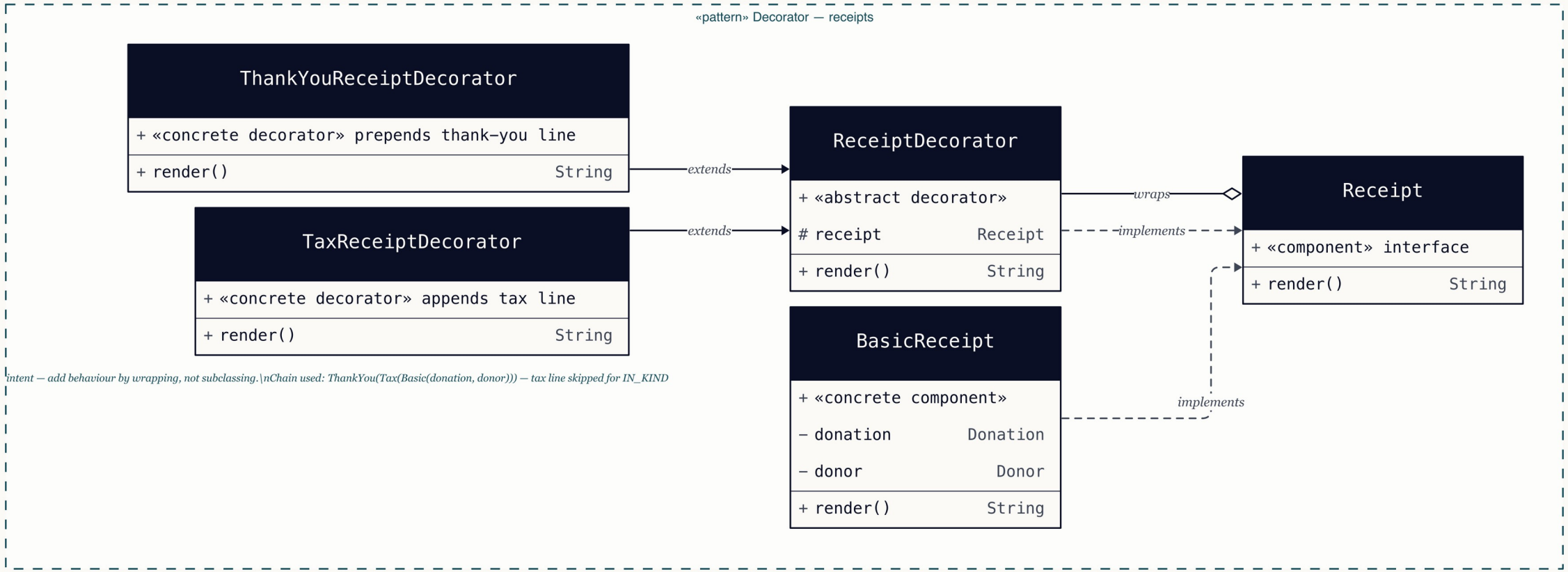
Where — com.resala.beneficiary.policy.ByNeed · Equal · FifoDistributionPolicy

Subjects notify; observers react



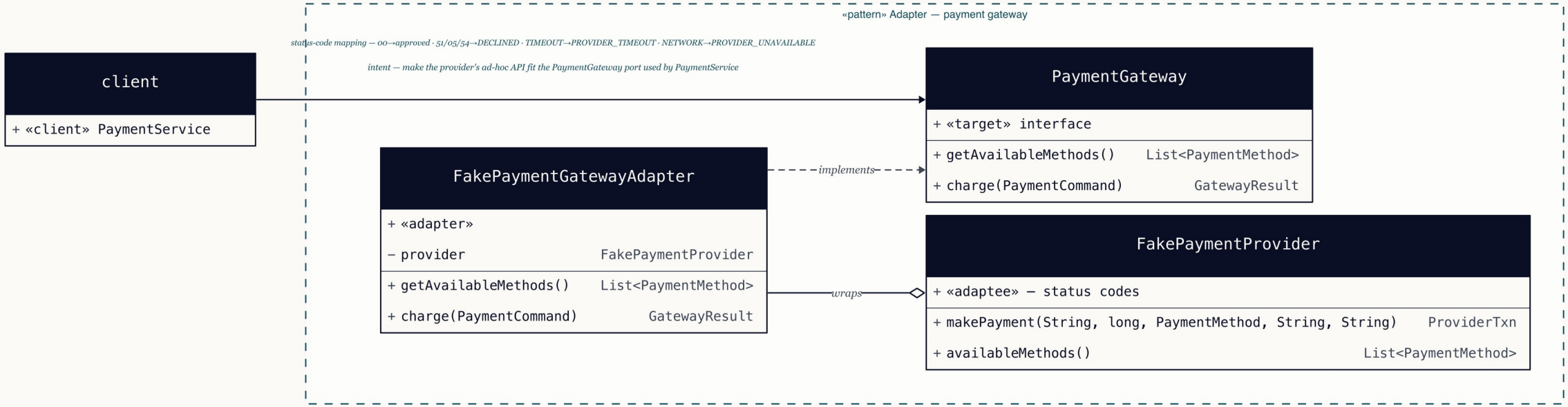
Where — com.resala.notification (DonationSubject · EventSubject · FeedObserver) · services implement subjects

Receipts composed at runtime



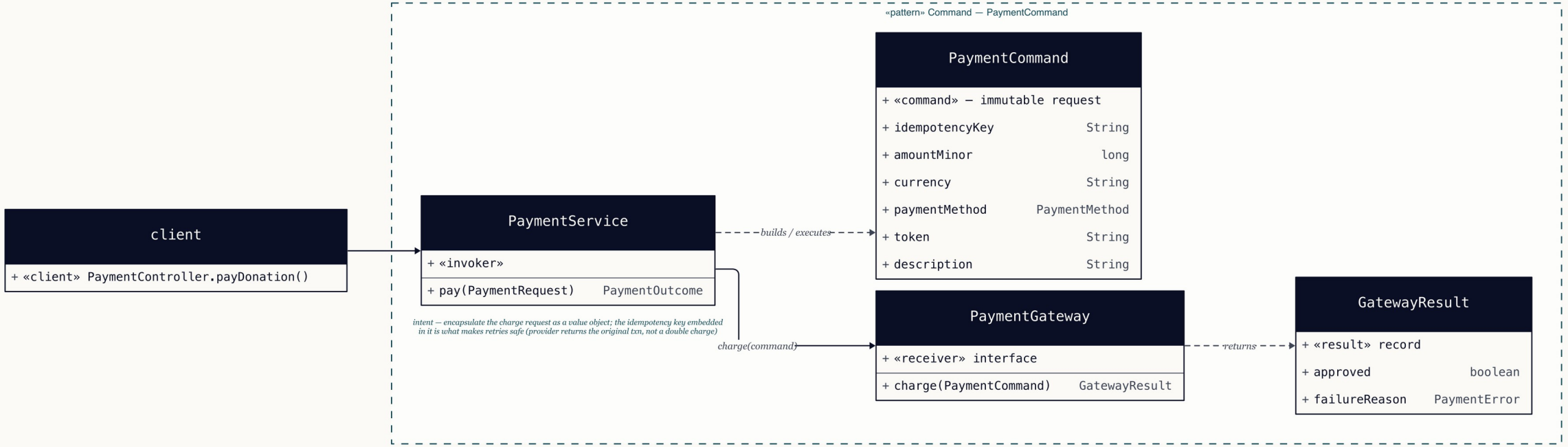
Where — com.resala.donation.Receipt · BasicReceipt · Tax / ThankYouReceiptDecorator

Provider’s API, our port



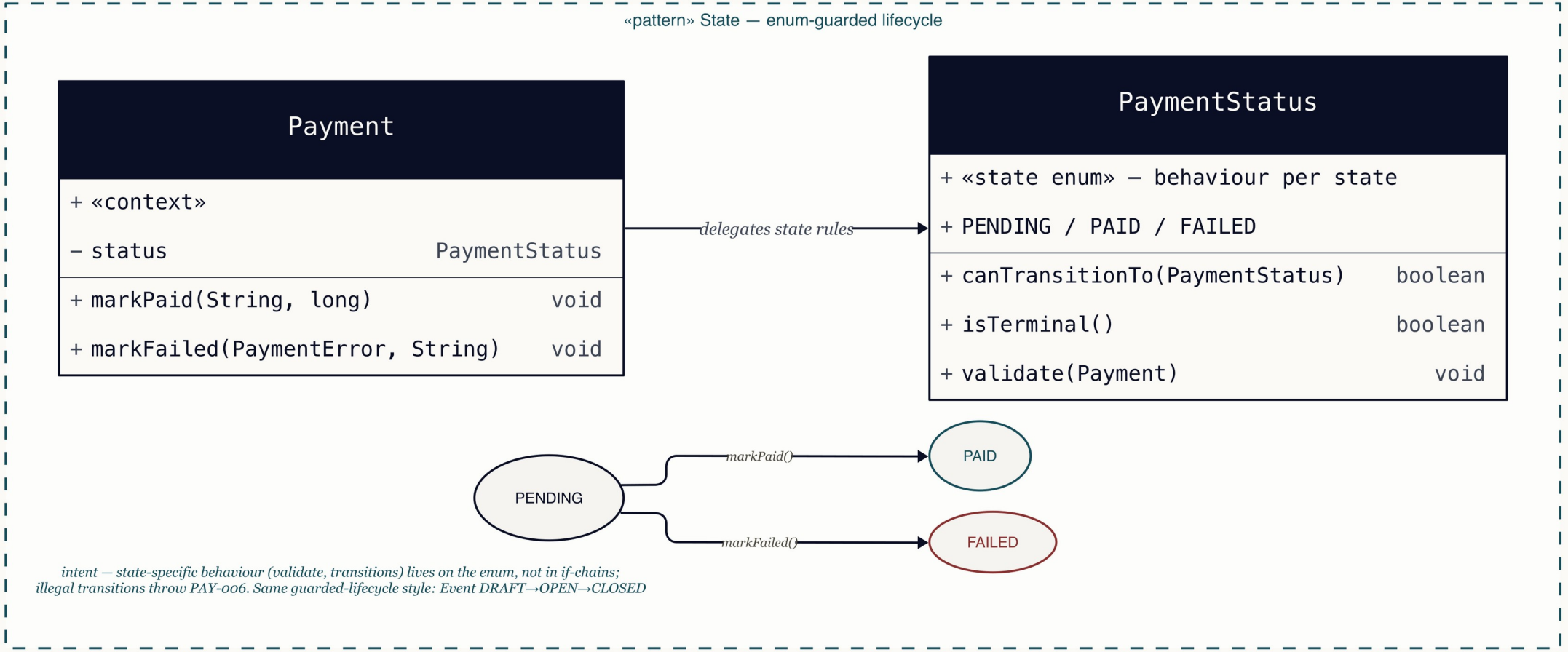
Where — com.resala.payment.FakePaymentGatewayAdapter ⟶ FakePaymentProvider

The charge, carried as a value



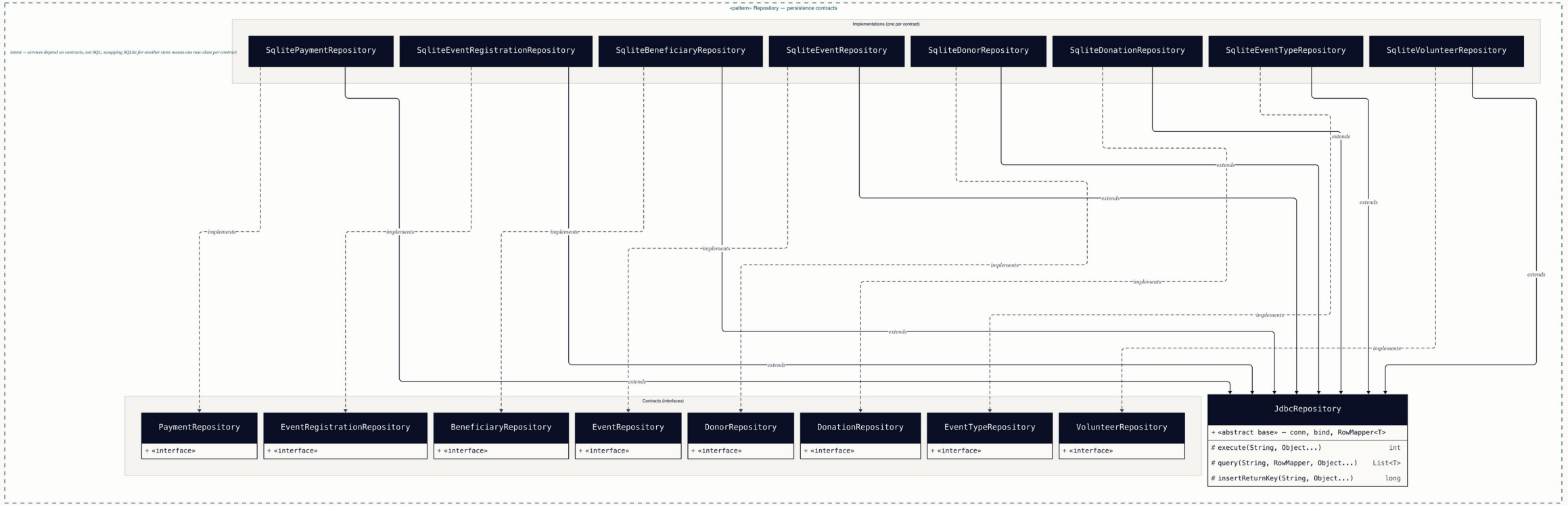
Where — com.resala.payment.PaymentCommand → gateway.charge(...) · idempotency key = retry safety

Transitions guarded by the enum



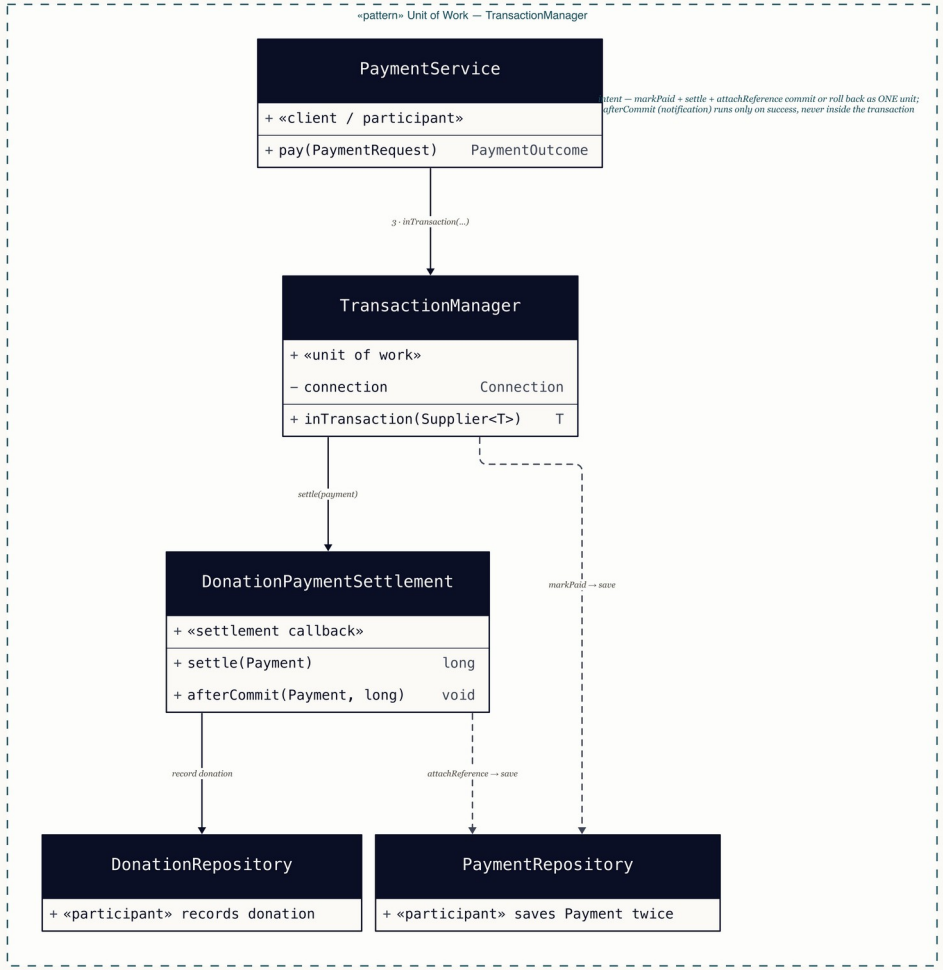
Where — com.resala.payment.PaymentStatus (PENDING → PAID / FAILED · PAY-006 on illegal moves)

Contracts in, SQL out of sight



*Where — com.resala.{donation,payment,event,volunteer,beneficiary}.*Repository · JdbcRepository*

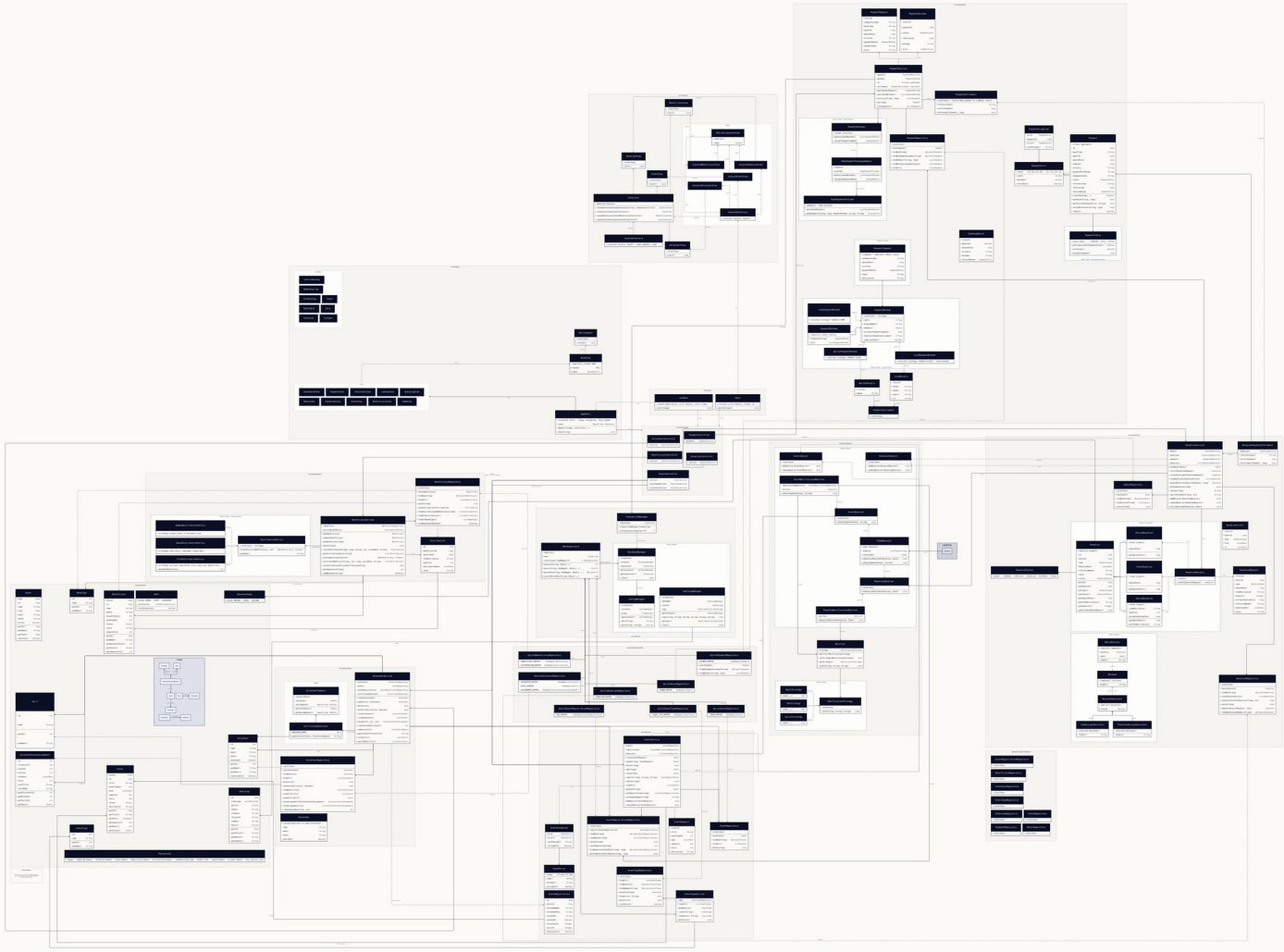
Settle atomically or not at all



Where — com.resala.core.TransactionManager · PaymentService · DonationPaymentSettlement.settle()

THE WHOLE MAP

Every class, one canvas



How to read this

· All 130+ classes, grouped by package.

· Same notation as the module pages.

· Text is unreadable at this size by design — this is the poster, not the reading copy.

· The slides 5–27 are the reading copies.

· Print the companion PDF page as a wall chart — or zoom the vector SVG.

Four files, one truth

output/Resala-Charity-UML-Full.svg
Complete class map — vector, infinite zoom, selectable text

output/Resala-Charity-UML.pptx
This deck — every module and pattern on its own slide

output/Resala-Charity-UML.pdf
Same deck, print-ready (converted from the .pptx)

output/Resala-Charity-UML-Full-poster.pdf
The all-system map as a single printable wall-chart page

REGENERATE

./build-diagrams → SVG + PNG + PDF for every diagram (incl. 99-full, assembled from the modules)
python3 diagrams/assemble-full.py → rebuild only the all-system map from module sources